Reviewer Pack — Minimal Local Cohesion Index in Quandles and SAT Clause Comp…

Entry d3e01901cb9f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 13:35:45 UTC

Minimal Local Cohesion Index in Quandles and SAT Clause Complexity

Entry ID: `d3e01901cb9f` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-08 13:35:45 UTC

1. Conjecture

Field A (mathematical branch): Algebra (Quandles) **Field B** (complexity object): Boolean Satisfiability (Clause Complexity)

Statement:

> The minimal local cohesion index of a quandle associated with an instance of size n ≤ 40 in SAT clause complexity is upper-bounded by the exponential of the maximum number of clauses that can be satisfied simultaneously, i.e., mci(Q) = O(2^(max_clauses_satisfiable)), where mci(Q) is the minimal local cohesion index of quandle Q.

Rationale (proposer's reasoning):

> Quandles are a generalization of groups and have been studied in algebraic combinatorics. The local cohesion index captures the interconnectedness within a structure, which could be related to the interdependencies between clauses in SAT instances. This conjecture could expose a new way to understand clause complexity by linking it with an algebraic structure.

Taxonomy category: `Algebra_BooleanSatisfiability` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `c9d8b75a7f866645`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> If for all generated quandles Q of size n ≤ 40, the minimal local cohesion index mci(Q) is less than or equal to twice the maximum number of clauses that can be satisfied simultaneously (2^(max_clauses_satisfiable)), then the conjecture is supported. If any quandle Q has a mci(Q) greater than 2^(max_clauses_satisfiable), the conjecture is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 11 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal Local Cohesion Index AND Quandles - Quandle SAT Clause Complexity - Algebraic Structures IN BOOLEAN Satisfiability

Top relevant hits considered: - [http://arxiv.org/abs/2112.02974v1] Randomness in the choice of neighbours promotes cohesion in mobile animal groups - [http://arxiv.org/abs/cond-mat/9903006v2] Cohesion and conductance of disordered metallic point contacts - [http://arxiv.org/abs/2603.27220v1] Cohesion-Sensitive Power Indices: Representation Results for Banzhaf and Shapley Values - [http://arxiv.org/abs/1908.01624v1] Learned Clause Minimization in Parallel SAT Solvers - [http://arxiv.org/abs/2207.13577v2] Scalable Proof Producing Multi-Threaded SAT Solving with Gimsatul through Sharing instead of Copying Clauses - [http://arxiv.org/abs/2012.15478v3] N-quandles of links - [http://arxiv.org/abs/1103.5740v2] Generating and Searching Families of FFT Algorithms - [http://arxiv.org/abs/2206.00903v2] Satisfiability of Quantified Boolean Announcements

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_quandle(n):
        quandle = []
        for i in range(n):
            row = [random.randint(0, n-1) for _ in range(n)]
            quandle.append(row)
        return quandle

    def apply_quandle(quandle, x):
        n = len(quandle)
        result = []
        for i in range(n):
            result.append(quandle[x][i])
        return result
```

```
def max_clauses_satisfiable(clauses):
    n = len(clauses[0])
    if not clauses:
        return 0
    max_sat = 0
    for mask in range(1 << n):
        satisfied = True
        for clause in clauses:
            if all((mask >> j) & 1 == (x < 0 and -x or x) % n in clause for x in clause):
                continue
            else:
                satisfied = False
                break
        if satisfied:
            max_sat += 1
    return max_sat

def minimal_local_cohesion_index(quandle):
    n = len(quandle)
    index = 0
    for i in range(n):
        for j in range(i+1, n):
            if quandle[i][j] == quandle[j][i]:
                index += 1
    return index

def generate_sat_instance(n):
    clauses = []
    for _ in range(n):
        clause = random.sample(range(1, n+1), random.randint(1, n))
        clauses.append(clause)
    return clauses

n_values = [5, 10, 15, 20, 30, 40]
metric_values = []
instances_tested = 0
n_max = 0
conjecture_holds = True
counterexample = ""

for n in n_values:
    for _ in range(5):
        quandle = generate_quandle(n)
        clauses = generate_sat_instance(n)
        mci = minimal_local_cohesion_index(quandle)
        max_sat = max_clauses_satisfiable(clauses)
        metric_values.append(mci <= 2**max_sat)
        instances_tested += 1
        n_max = max(n_max, n)

mean_metric_value = sum(metric_values) / len(metric_values)
support_fraction = sum(metric_values) / len(metric_values)

if all(metric_values):
    return {
```

```
            "metric_name": "mci <= 2^(max_clauses_satisfiable)",
            "metric_value": mean_metric_value,
            "instances_tested": instances_tested,
            "n_max": n_max,
            "conjecture_holds": True,
            "counterexample": ""
        }
    else:
        for i, mci in enumerate(metric_values):
            if not mci:
                counterexample = f"mci({i}) > 2^(max_clauses_satisfiable)"
                break
        return {
            "metric_name": "mci <= 2^(max_clauses_satisfiable)",
            "metric_value": mean_metric_value,
            "instances_tested": instances_tested,
            "n_max": n_max,
            "conjecture_holds": False,
            "counterexample": counterexample
        }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fract
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_hol
        print(f"RESULT: FALSIFIED counterexample=\"{r['counterexample']}\" first_failing_seed={fir
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fc1a1ce7.py", line 117, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fc1a1ce7.py", line 80, in run_trial
    max_sat = max_clauses_satisfiable(clauses)
```

```
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fc1a1ce7.py", line 43, in max_clauses_s
    if all((mask >> j) & 1 == (x < 0 and -x or x) % n in clause for x in clause):
       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fc1a1ce7.py", line 43, in <genexpr>
    if all((mask >> j) & 1 == (x < 0 and -x or x) % n in clause for x in clause):
                   ^
NameError: name 'j' is not defined
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which means we cannot verify the conjecture's support or falsification. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12502
2	preregistration	ollama_remote	glm4:latest	0	0	9710
3	novelty	ollama_remote	glm4:latest	0	0	8219
4	novelty	ollama_remote	glm4:latest	0	0	11597
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	41945
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11788
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10626
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12030
9	judge	ollama_remote	glm4:latest	0	0	11792

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 130207 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/d3e01901cb9f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d3e01901cb9f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/d3e01901cb9f.tar.gz` (if generated)